

Класи: поліморфізм, інтерфейси, патерни проєктування

Заняття 14 (3.2)

C++ для військових технологій

Лектор
Євген Гаврилюк

r_d

ПЛАН ЗАНЯТТЯ



- Поліморфізм: virtual, override, vtable
- Чисто віртуальні функції та абстрактні класи
- Інтерфейси: ITargetProvider, IBallisticSolver
- Патерни: Стратегія, Фабрика, Будівельник

VIRTUAL vs ЗВИЧАЙНИЙ МЕТОД

Два методи в базовому класі: один virtual, інший — ні. Різниця видна при виклику через Device*.

Базовий клас

```
class Device {
public:
    const char* getName() {
        return "Device";
    }
    virtual const char* getType() {
        return "generic";
    }
    virtual ~Device() {}
};
```

Похідний клас

```
class UAV : public Device {
public:
    const char* getName() {
        return "UAV";
    }
    const char* getType() override {
        return "quadcopter";
    }
};
```

Виклик

```
UAV mavic;
Device* d = &mavic;

printf("%s\n", d->getName());
printf("%s\n", d->getType());
```

Вивід

Device
quadcopter

getName() — не virtual → виклик за типом вказівника (Device)
getType() — virtual → виклик за реальним типом об'єкта (UAV)

НАВІЩО ПОЛІМОРФІЗМ?

```
Device* fleet[3];  
fleet[0] = new UAV();  
fleet[1] = new GroundStation();  
fleet[2] = new UAV();  
  
for (int i = 0; i < 3; i++) {  
    printf("[%d] %s\n", i, fleet[i]->getType());  
    fleet[i]->update(0.1f);  
}  
  
for (int i = 0; i < 3; i++)  
    delete fleet[i];
```

Один цикл — різна поведінка

Вивід:

[0] quadcopter
[1] ground-station
[2] quadcopter

Кожен об'єкт викликає **свій** getType() та **свій** update().

Цикл не знає і не хоче знати конкретні типи.

Три головні переваги

01

Розширюваність Додав
Relay, Jammer — цикл
працює без змін.

02

Підміна AnalyticalSolver →
TableSolver — виклик
solve() не змінюється.

03

Тестування TestProvider замість
JsonProvider — тестуєш без
зовнішніх залежностей.

VIRTUAL / OVERRIDE

Базовий клас

```
class Device {
protected:
    Coord position;
    float batteryPercent;
    char callsign[32];
public:
    virtual const char* getType() const
        { return "generic"; }
    virtual void update(float dt)
        { batteryPercent -= 0.01f * dt; }
    bool isOperational() const
        { return batteryPercent > 0; }
    virtual ~Device() {}
};
```

virtual Метод може бути перевизначений у нащадку. Рішення — в runtime.

override «Я перевизначаю метод бази». Компілятор перевірить сигнатуру.

Похідний клас

```
class UAV : public Device {
    float altitude;
public:
    UAV(float x, float y, float alt,
        const char* cs)
        : Device(x,y,100,cs), altitude(alt){}

    const char* getType() const override
        { return "quadcopter"; }
    void update(float dt) override {
        consume(0.5f * dt);
        fly(dt);
    }
};
```

Без virtual isOperational() — звичайний метод, однаковий для всіх нащадків.

virtual ~Device() Деструктор **ЗАВЖДИ virtual у базі!** Є virtual метод → є virtual деструктор.

VTABLE — ЯК ПРАЦЮЄ ПІД КАПОТОМ

Кожен об'єкт з virtual методами містить прихований vptr → вказівник на таблицю віртуальних функцій.

Об'єкт Device d

vptr →

position

batteryPercent

callsign

Об'єкт UAV u

vptr → (інша таблиця!)

Device vtable

[0] Device::getType

[1] Device::update

[2] Device::~Device

UAV vtable

[0] UAV::getType

[1] UAV::update

[2] UAV::~UAV

Як працює виклик

```
Device* d = new UAV;  
d->getType();
```

1. Читає vptr з об'єкта
2. vptr вказує на UAV vtable
3. Бере [0] → UAV::getType
4. Викликає UAV::getType()

Ціна: один додатковий indirect call через вказівник. На сучасному CPU — мінімальна.

ЧИСТО ВІРТУАЛЬНІ ФУНКЦІЇ

```
class Device {
public:
    virtual const char* getType() const = 0;
    virtual void update(float dt) = 0;
    virtual ~Device() {}
};

// Device d; // ПОМИЛКА! абстрактний клас

class UAV : public Device {
public:
    const char* getType() const override
    { return "UAV"; }
    void update(float dt) override { /*...*/ }
};

UAV u; // OK — всі = 0 реалізовані
```

= 0 → абстрактний клас

virtual f() = 0

Метод без реалізації в базі. Нащадок
ЗОБОВ'ЯЗАНИЙ реалізувати.

Абстрактний клас

Клас з ≥ 1 чисто віртуальним методом. Не
можна створити об'єкт.

Навіщо?

Визначає контракт: «кожен Device
ПОВИНЕН мати getType() і update()».
Компілятор перевірить.

Коли робити метод чисто віртуальним?

Коли базова реалізація не має сенсу. Device::getType() — а який тип? Невідомо. Device::update() — а що оновлювати? Нічого. Якщо є розумна базова реалізація — залиште звичайний virtual (нащадок може override, а може й ні).

ІНТЕРФЕЙС = ТІЛЬКИ ЧИСТІ ВІРТУАЛЬНІ

Інтерфейс — клас без даних, тільки чисто віртуальні методи. Визначає ЩО робити, не ЯК.

```
// Провайдер даних про цілі
class ITargetProvider {
public:
    virtual int getTargetCount() = 0;
    virtual Target getTarget(int idx) = 0;
    virtual ~ITargetProvider() {}
};

// Солвер балістичної задачі
class IBallisticSolver {
Public:
    virtual BallisticTrajectory solve(float speed, float alt,
const AmmoParams& ammo
    ) = 0;
    virtual ~IBallisticSolver() {}
};
```

Конвенція

Префікс I — ITargetProvider, IBallisticSolver. В C++ це лише конвенція (не як в Java/C#).

Без полів даних

Тільки = 0 методи + virtual деструктор.

Множинне наслідування

```
class UAV : public Device,
           public ISensor
Клас реалізує кілька інтерфейсів одночасно.
```

«interface»
ITargetProvider

JsonProvider

TestProvider

«interface»
IBallisticSolver

AnalyticalSolver

TableSolver

РЕАЛІЗАЦІЇ — ITARGETPROVIDER

JsonTargetProvider

```
class JsonTargetProvider
: public ITargetProvider {
    Target targets[64];
    int count;
public:
    JsonTargetProvider(const char* path) {
        count = parseJson(path, targets, 64);
    }
    int getTargetCount() override
    { return count; }
    Target getTarget(int i) override
    { return targets[i]; }
};
```

TestTargetProvider

```
class TestTargetProvider
: public ITargetProvider {
    Target data[2] = {
        {{300,200}, {0,0}, "Tank-1"},
        {{500,100}, {5,0}, "APC-2"}
    };
public:
    int getTargetCount() override { return 2; }
    Target getTarget(int i) override
    { return data[i]; }
};
```

Використання

```
void engage(ITargetProvider* prov,
            IBallisticSolver* solver) {
    for (int i = 0;
         i < prov->getTargetCount(); i++){
        Target t = prov->getTarget(i);
        auto dp = solver->solve(
            dronePos, t.pos, alt, ammo);
    }
}
```

Головна перевага:

engage() працює з будь-яким провайдером.

Тестування:

TestProvider — тестуєш без файлів.

Розширення:

MavlinkProvider, SerialProvider —
новий клас, існуючий код не чіпаємо.

РЕАЛІЗАЦІЇ — IBALLISTICSOLVER

AnalyticalSolver

```
class AnalyticalSolver
: public IBallisticSolver {
public:
    DropPoint solve(Coord drone, Coord tgt,
        float alt, const AmmoParams& a
    ) override {
        float t = sqrt(2.0f * alt / 9.81f);
        float drift = a.dragCoeff * t;
        return {tgt.x - drift, tgt.y};
    }
};
```

Два підходи до однієї задачі

AnalyticalSolver

Формула. Швидко, але неточно.

TableSolver

Таблиця відстрілів + інтерполяція. Точніше, бо побудована за реальними даними.

Той самий інтерфейс!

TableSolver

```
class TableSolver
: public IBallisticSolver {
    BallisticTable* table;
public:
    TableSolver(BallisticTable* t)
        : table(t) {}
    DropPoint solve(Coord drone, Coord tgt,
        float alt, const AmmoParams& a
    ) override {
        return table->interpolate(
            alt, dist(drone, tgt), a);
    }
};
```

Це і є паттерн Стратегія!

Стратегія = інтерфейс алгоритму + кілька реалізацій.
Вибір — в runtime.

Функція отримує IBallisticSolver* — не знає, аналітика це чи таблиця. Можна підмінити на льоту.

ПАТТЕРН: СТРАТЕГІЯ

Визначити сімейство алгоритмів, інкапсулювати кожен, зробити їх взаємозамінними.

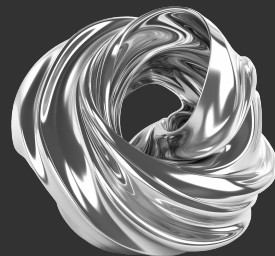
```
class Mission {
    IBallisticSolver* solver;    // стратегія
    ITargetProvider* targets;
public:
    Mission(IBallisticSolver* s, ITargetProvider* t) : solver(s), targets(t) {}

    DropPoint computeDrop(int idx, Coord dronePos, float alt, const AmmoParams& ammo) {
        Target t = targets->getTarget(idx);
        return solver->solve(dronePos, t.pos, alt, ammo);
    }
    int getTargetCount() { return targets->getTargetCount(); }
    void changeSolver(IBallisticSolver* s) { solver = s; }
};
```

```
AnalyticalSolver    analytical;
TableSolver         table(&ballisticData);
JsonTargetProvider  provider("targets.json");

Mission mission(&analytical, &provider);
auto dp = mission.computeDrop(0, dronePos, 100, vog17);    // аналітика

mission.changeSolver(&table);                                // підміна на таблицю
dp = mission.computeDrop(0, dronePos, 100, vog17);          // той самий виклик
```



ПАТТЕРН: ФАБРИКА

Фабрика створює об'єкти потрібного типу, не розкриваючи логіку вибору.

```
enum class SourceType { JSON, SERIAL, TEST };

ITargetProvider* createProvider(
    SourceType type, const char* param) {
    switch (type) {
        case SourceType::JSON:
            return new JsonTargetProvider(param);
        case SourceType::SERIAL:
            return new SerialTargetProvider(param);
        case SourceType::TEST:
            return new TestTargetProvider();
        default: return nullptr;
    }
}

// Використання:
auto* prov = createProvider(
    SourceType::JSON, "targets.json");
delete prov;
```

Навіщо Фабрика?

1. Централізація

Одне місце, де вирішується, який клас створити.

2. Конфігурація

Тип провайдера можна читати з конфігу, командного рядка, JSON.

3. Тестування

createProvider(TEST, "") — тестуєш без зовнішніх залежностей.

Увага: Фабрика повертає вказівник → хтось **ПОВИНЕН** зробити delete. В C++11+ краще unique_ptr.

Стратегія vs Фабрика: Стратегія — підміна алгоритму (solver). Фабрика — створення потрібного об'єкта (provider). Часто працюють разом: Фабрика створює, Стратегія використовує.

ПАТТЕРН: БУДІВЕЛЬНИК (BUILDER)

Покроково збирає складний об'єкт. Замість конструктора з 10 параметрами — зрозумілий ланцюжок.

Конфігурація

```
struct MissionConfig {  
    Coord startPos;  
    float altitude;  
    int maxTargets;  
    SourceType loaderType;  
    char loaderParam[256];  
    bool useTableSolver;  
};
```

Builder клас

```
class MissionBuilder {  
    MissionConfig cfg{};  
public:  
    MissionBuilder& startAt(float x, float y)  
    { cfg.startPos={x,y}; return *this; }  
    MissionBuilder& altitude(float a)  
    { cfg.altitude=a; return *this; }  
    MissionBuilder& maxTargets(int n)  
    { cfg.maxTargets=n; return *this; }  
    MissionBuilder& jsonProvider(const char* p)  
    { cfg.loaderType=SourceType::JSON;  
      strncpy(cfg.loaderParam,p,255);  
      return *this; }  
    MissionConfig build() { return cfg; }  
};
```

Використання

```
auto cfg = MissionBuilder()  
    .startAt(0, 0)  
    .altitude(100)  
    .maxTargets(32)  
    .jsonProvider("targets.json")  
    .build();
```

Навіщо Builder?

Замість:

Mission(0,0,100,32,JSON,"t.json")

— незрозуміло що є що.

return *this

Метод повертає посилання на себе → можна chaining:

.startAt().altitude().build()

Опціональні параметри

Частина методів можна не викликати — будуть дефолти.

КОЛИ ЩО ВИКОРИСТОВУВАТИ?

Паттерн	Коли?	Приклад з курсу
Стратегія	Кілька алгоритмів для однієї задачі. Треба міняти на льоту.	IBallisticSolver: AnalyticalSolver vs TableSolver. Місія не знає деталей.
Фабрика	Створення об'єкта залежить від параметра (enum, config, string).	createProvider(JSON, path) → повертає JsonTargetProvider*. Тип вирішується в одному місці.
Будівельник	Складний об'єкт з багатьма параметрами. Частина — опціональна.	MissionBuilder() .startAt(0,0).altitude(100) .jsonLoader("t.json").build()

Ці паттерни часто комбінуються: Фабрика створює потрібний Solver (Стратегію), Будівельник збирає Mission із Фабрикою та Стратегією всередині.
Головне правило: не ускладнюй без потреби. Якщо switch вирішує задачу — не потрібна Фабрика. Якщо один алгоритм — не потрібна Стратегія.

ПОШИРЕНІ ПОМИЛКИ

Забули `virtual` деструктор

```
class Device { public:  
    ~Device() {} // НЕ virtual!  
};  
Device* d = new UAV();  
delete d; // ~UAV() НЕ ВИКЛИКАЄТЬСЯ!
```



Забули `override`

```
class UAV : public Device {  
    const char* getType() { // забув const!  
        return "UAV";  
    }  
    // Без override — тиха помилка  
    // З override — ПОМИЛКА КОМПІЛЯЦІЇ!  
};
```



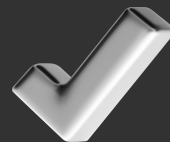
Object slicing

```
UAV uav("Mavic-1");  
Device d = uav; // COPY! обрізається!  
d.getType();    // → "Device", не "UAV"  
// Поліморфізм — ТІЛЬКИ через * або &
```



Як правильно

1. Деструктор бази ЗАВЖДИ `virtual`
2. ЗАВЖДИ пишіть `override`
3. Поліморфізм — через `Device*` / `Device&`
4. Ніколи `Device d = uav` (обрізає!)



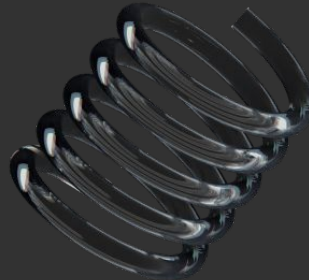
Правило курсу: Компілюйте з `-Wall -Wextra -Werror`.

Якщо є `virtual` — є `virtual ~Destructor()`. Якщо є `override` — є `override` скрізь.

Clang-Tidy: `-Wsuggest-override -Wnon-virtual-dtor` — ловить ці помилки автоматично.

ПОВНИЙ ПРИКЛАД — МІСІЯ

```
int main() {  
    JsonTargetProvider provider("targets.json");  
    AnalyticalSolver analytical;  
    TableSolver table(&ballisticData);  
  
    Mission mission(&analytical, &provider);  
  
    AmmoParams vog17 = {"VOG-17", 0.28f, 0.003f};  
    DropPoint dp = mission.computeDrop(  
        0, {0, 0}, 100.0f, vog17  
    );  
    printf("Analytical: (%.1f, %.1f)\n",  
        dp.x, dp.y);  
  
    mission.changeSolver(&table);  
    dp = mission.computeDrop(  
        0, {0, 0}, 100.0f, vog17  
    );  
    printf("Table: (%.1f, %.1f)\n",  
        dp.x, dp.y);  
    return 0;  
}
```



Що відбувається

1. Створюємо provider + два solver
2. Збираємо Mission (Стратегія)
3. Обчислюємо analytical
4. Підміна solver → table
5. Обчислюємо table — виклик той самий!

Вивід

Analytical: (297.2, 200.0)

Table: (295.8, 199.4)

Різний результат —
різний алгоритм, той самий
інтерфейс IBallisticSolver.

Q&A





ЗАВЖДИ Є КУДИ
ЗРОСТАТИ